

El Perceptrón

curso
Deep Learning

Prof. M.A.Valle
E-MAIL: mvalle@uft.cl

Modeling the basic structure

El **perceptrón** se define como la unidad básica de construcción de una red neuronal. En ML, un perceptrón es un algoritmo de aprendizaje supervisado para clasificadores binarios.

Modeling the basic structure

El **perceptrón** se define como la unidad básica de construcción de una red neuronal. En ML, un perceptrón es un algoritmo de aprendizaje supervisado para clasificadores binarios.

El **perceptrón** se puede entender como una neurona que toma múltiples entradas y produce una salida intentando **minimizar el error**.

El proceso de alterar los pesos se denomina error **backpropagation**. Altera los pesos (sinapsis) basados en la salida actual de la red con respecto a una entrada, como la diferencia entre la salida actual y la salida deseada.

Modeling the basic structure

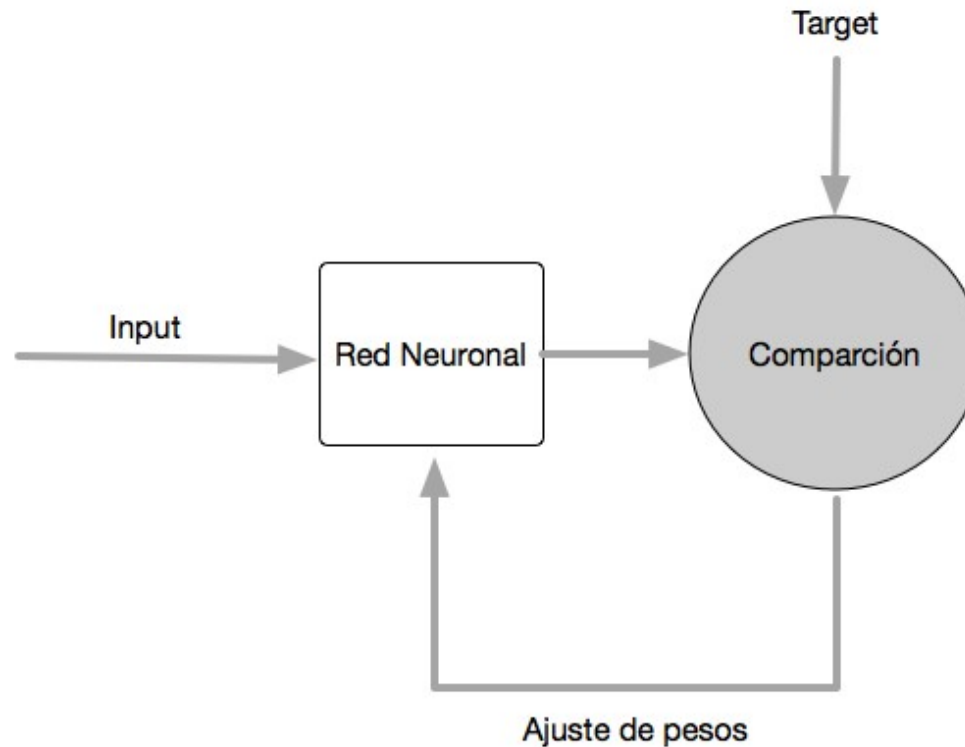
El **perceptrón** se define como la unidad básica de construcción de una red neuronal. En ML, un perceptrón es un algoritmo de aprendizaje supervisado para clasificadores binarios.

El **perceptrón** se puede entender como una neurona que toma múltiples entradas y produce una salida intentando **minimizar el error**.

El proceso de alterar los pesos se denomina error **backpropagation**. Altera los pesos (sinapsis) basados en la salida actual de la red con respecto a una entrada, como la diferencia entre la salida actual y la salida deseada.

Veremos que el perceptrón sólo es capaz de reconocer sólo funciones lineales separables (**linearly separable functions**).

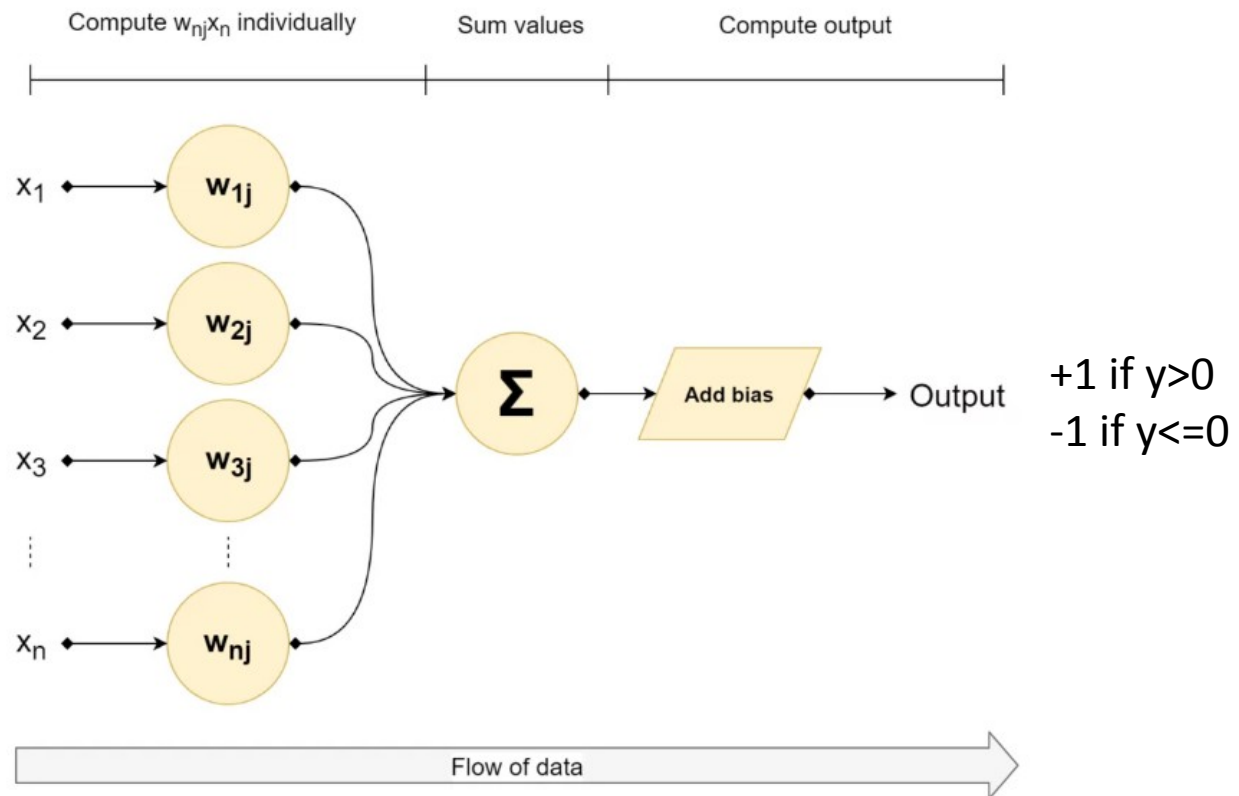
Modeling the basic structure



La idea del aprendizaje con un perceptrón.

Modeling the basic structure

En su forma más simple, un perceptrón posee una neurona que acepta entradas y produce una salida.



Re-acondicionado de <https://www.machinecurve.com/>

Modeling the basic structure

Aquí, $x_1, x_2, \dots, x_n$ es el set de inputs de una instancia y x_0 es el bias. La salida y es la suma de los productos $w_i x_i$. A esta suma se le aplica una función de activación, por ejemplo, signum:

si $y > 0$, $y = 1$

si $y \leq 0$, $y = -1$

El bias es constante y está asociado al peso w_0 .

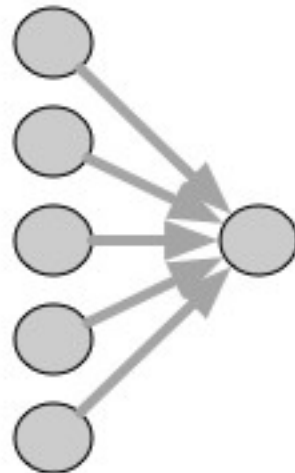
Note que no tenemos backpropagation por ahora, de modo que no hay update de los pesos.

Así, un perceptrón es una **función de clasificación lineal** separando las salidas en alguna de dos categorías: +1 o -1.

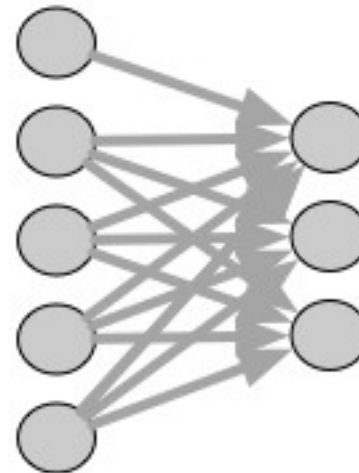
Modeling the basic structure....PERO

También podríamos tener otro tipo de perceptrón, llamado el multi-class perceptron, que puede clasificar múltiples etiquetas para un animal como perro, gato o canario.

Perceptrón simple



Perceptrón multiclass



El perceptrón como un clasificador lineal

Recordemos: Un simple perceptrón es una simple capa de una neurona que actúa como un clasificador lineal: Produce dos tipos de salida ON (activo) o OFF (inactivo).

Esta decisión de ON o OFF la determina la **función de activación**.

$$y = \begin{cases} -1, & \text{if } w \cdot x \leq 0 \\ 1, & \text{if } w \cdot x > 0 \end{cases}$$

Vemos la interacción lineal aquí; la salida 'y' es LD de sus entradas.

Al principio, los pesos (w) tienen valores aleatorios.

En el ENTRENAMIENTO, usamos algunos ejemplos con su clasificación correcta para presentarlos a la red. Si la red se equivoca en la clasificación, entonces “ajustamos” los pesos para que esto no ocurra.

El perceptrón como un clasificador lineal - aprendizaje

1. Obtener x , y la salida y (real)

2. Si $f(x) = y$, exit, else **adjust**

3. **adjust**:

(a) $f(x) = \text{sgn}(\sum w \times x)$

(b) si $y = +1$, $f(x) = -1$, then $w \times x$ is too small (agrandar los w)

(c) si $y = -1$, $f(x) = +1$, then $w \times x$ is too large (achicar los w)

4. **learn**:

(a) make $w = w - x$ si $f(x) = +1$, $y = -1$ (achicamos w)

(b) make $w = w + x$ si $f(x) = -1$, $y = +1$ (agrandamos w)

(c) o simplemente en resumen: $w = w + yx$ si $f(x) \neq y$

5. Repeat step 3 to 5 until $f(x) = y$

El perceptrón como un clasificador lineal

Lo esencial del aprendizaje está en el paso 5. En este paso podemos re-escribir:

$$w = w + \alpha yx \text{ if } f(x) \neq y \quad (1)$$

Donde la constante alpha es la [tasa de aprendizaje](#).

El bias también se actualiza como:

$$b = b + \alpha y \text{ if } f(x) \neq y$$

Si queremos ver implementado a mano este simple algoritmo, nos podemos dirigir a:

model_perceptron_byhand.ipynb (*búsquelo en github*)

El perceptrón como un clasificador lineal

Si la función de activación sgn (booleana) es linealmente separable, entonces, el perceptrón local pueden encontrar un set de pesos capaz de lograr el ajuste en un número finito de pasos.

Este es el **TEOREMA DEL PERCEPTRÓN** (perceptron theorem).

Aplicable a todos los perceptrones de una red, el cual modifica el vector de pesos sinápticos w .

El perceptrón como un clasificador lineal

Si la función de activación sgn (booleana) es linealmente separable, entonces, el perceptrón local pueden encontrar un set de pesos capaz de lograr el ajuste en un número finito de pasos.

Este es el **TEOREMA DEL PERCEPTRÓN** (perceptron theorem).

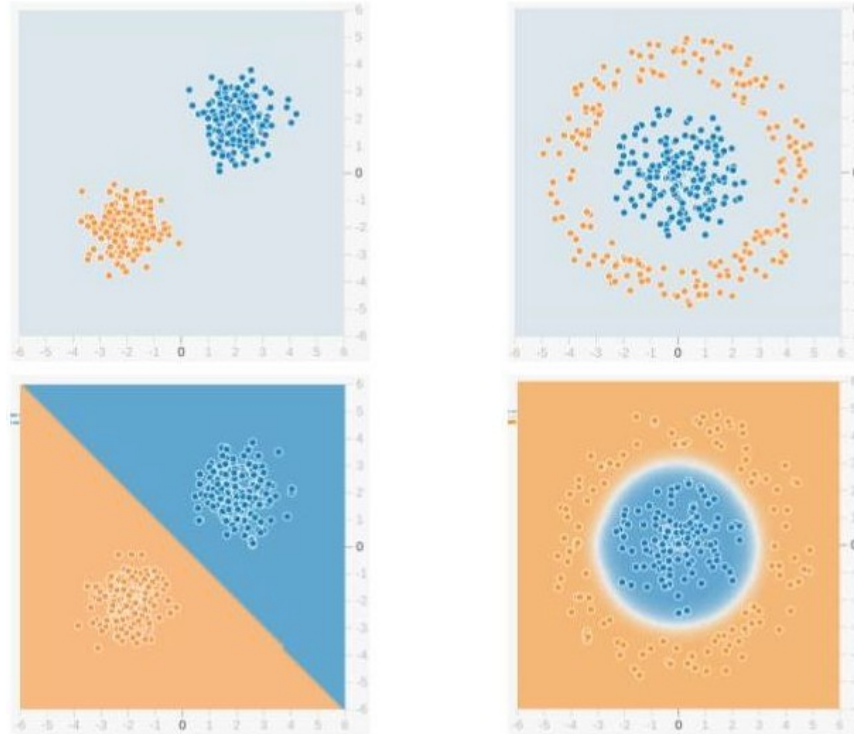
Aplicable a todos los perceptrones de una red, el cual modifica el vector de pesos sinápticos w .

What???

Pero que diablos es eso de linealmente separable????

Linear separation

La intuición:



En aprendizaje supervisado, como clasificación binaria, buscamos un modelo que nos discrimine una clase de otra.

Los puntos azules son de una clase, y los naranjas de la otra. La línea imaginaria que separa grupos se llama “**Decision Boundaries**” . La función de decisión es aprendida del set de etiquetas (muestras) que se llaman “set de datos de entrenamiento”.

Linear separation

¿cómo se produce el aprendizaje en el perceptrón?

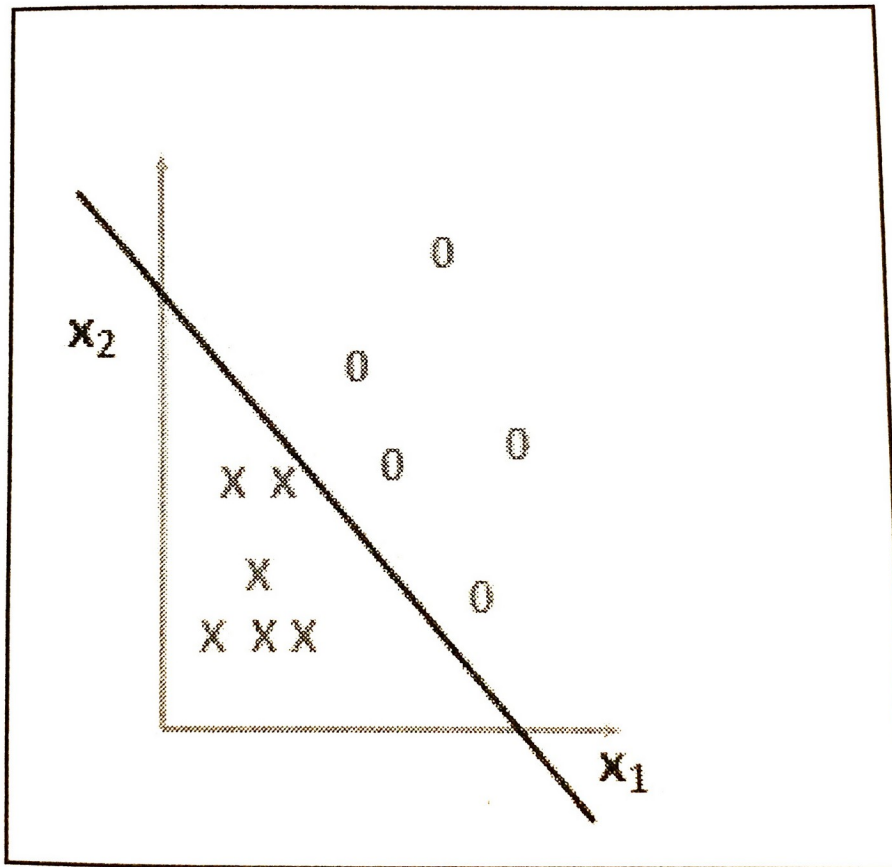
R:

La regla de aprendizaje del perceptrón.

(ver apunte [2.1 La regla de entrenamiento del Perceptrón.pdf](#))

Linear separation

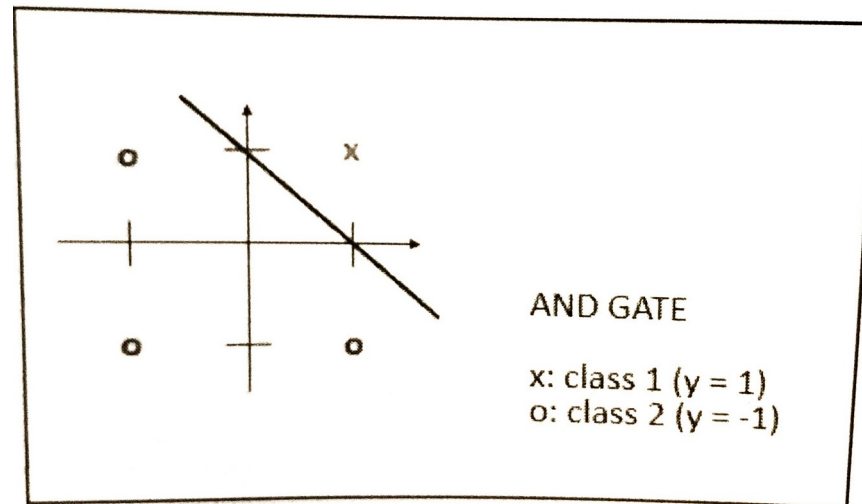
Cuando un set de salidas pueden ser discriminadas por una línea recta, entonces los valores de salida son “linealmente separables”. Hay un hiperplano que separa en el espacio de las entradas, aquellos en la condición de activos de los de la condición de inactivos.



La regla de decisión de la neurona booleana corresponde a un breakdown de los atributos de entrada operado por un hiperplano.

Linear separation ¿se puede modelar un AND?

x1	x2	y (AND gate)
1	1	1
1	0	0
0	1	0
0	0	0



Linear separation

Trabajo en clase : ¿se podría modelar un OR?

x1	x2	y (OR gate)
1	1	1
1	0	1
0	1	1
0	0	0

?

Linear separation

Trabajo en clase : ¿se podría modelar un XOR?

x1	x2	y (XOR gate)
1	1	0
1	0	1
0	1	1
0	0	0

?

Algunos ejemplos para comenzar

Comencemos a jugar con Python implementando un perceptron, utilizando un dataset real.

Nuestra primera aproximación también a Keras.

Si no tiene instalado JupyterLab en su computador, podemos hacer estos ejemplos en JupyterLab online aquí:

<https://jupyter.org/>

Y seleccionar "Try it in your browser"

(nota: puede tener problemas de acceso por medidas de seguridad de su browser)

Algunos ejemplos para comenzar

model_1neuron.ipynb

Consideremos un ejemplo muy simple con sólo 1 neurona. El modelo de esta red neuronal debe tomar un número cualquiera y convertirlo en negativo:



$$y = f(w, x) = -x$$

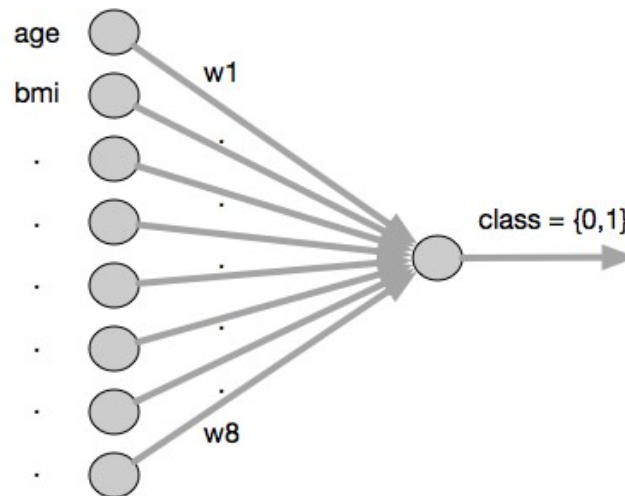
Preguntas de análisis:

- Qué valor debiera tener el peso sináptico?
- Es un MLP?
- Cuál es la capa escondida?
- Cuál debiera ser la función de activación?
- Cuál debiera ser la medida de desempeño para ajustar el w ?

Algunos ejemplos para comenzar

model_perceptron1.ipynb

Vamos a utilizar Keras para simular un perceptron.



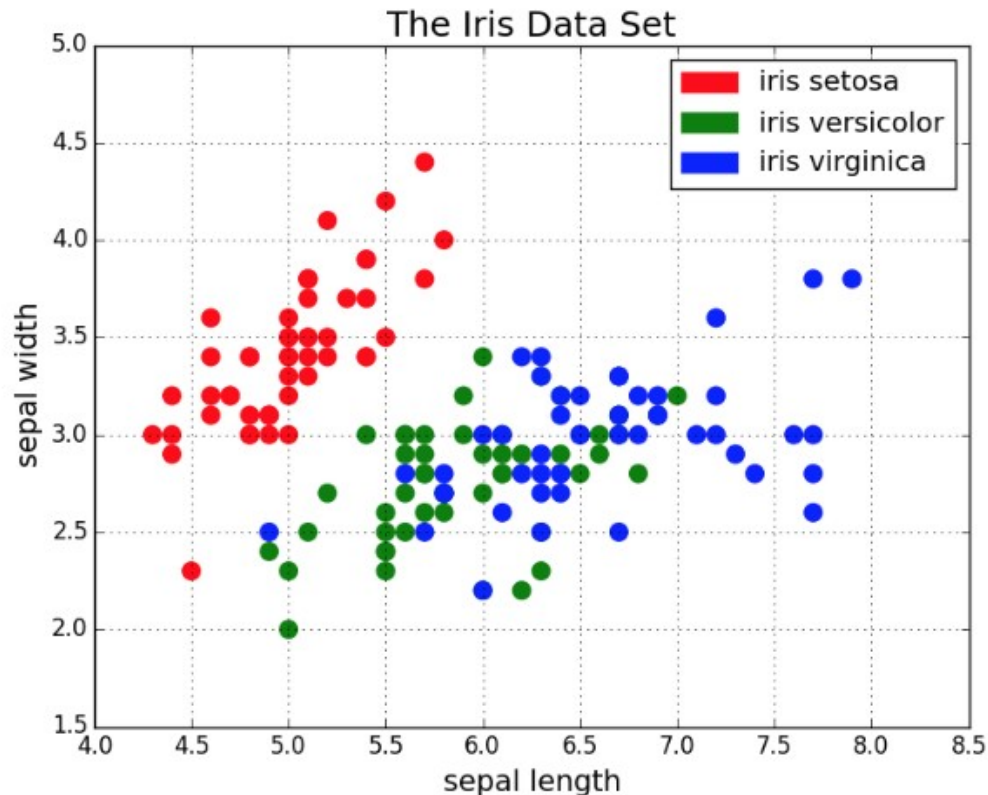
Preguntas de análisis:

- Qué valor debiera tener el peso sináptico?
- Es un MLP?
- Cuál es la capa escondida?
- Cuál debiera ser la función de activación?
- Cuál debiera ser la medida de desempeño para ajustar el w ?
- Qué interpretación le podemos dar a los pesos?

Algunos ejemplos para comenzar

`model_perceptron2.ipynb`

Vamos a utilizar Keras para simular un perceptron utilizando el archiconocido Iris dataset por qué otra vez????



Algunos ejemplos para comenzar

model_perceptron2.ipynb

Vamos a utilizar Keras para simular un perceptron.

¿cómo sería la estructura del perceptron?

Algunos ejemplos para comenzar

model_perceptron3.ipynb

Dado los resultados anteriores, trabajaremos un un perceptron que nos permita discriminar sólo entre setosa y versicolor.

Preguntas:

- Será mejor el desempeño del perceptron esta vez? Por qué?
- ¿Cómo sería la estructura del perceptron? Igual que antes?

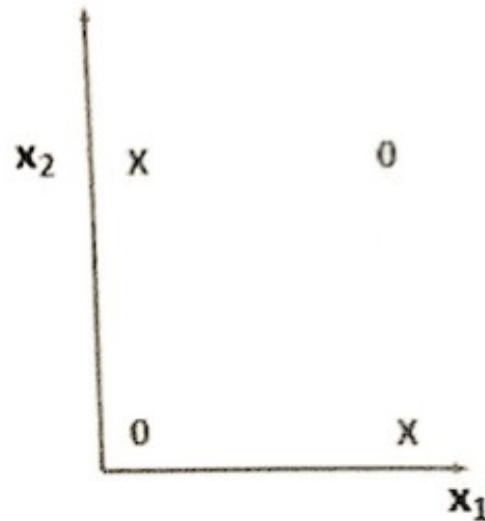
Nota: aquí vamos a importar el data set Iris.csv

Multi-layer Perceptron

Multi-layer Perceptron

Vimos que el AND y el OR son **linealmente separables** y que el perceptron lo puede reconocer.

Pero también nos dimos cuenta que no todas funciones son separables linealmente, por ejemplo, el XOR:



XOR GATE

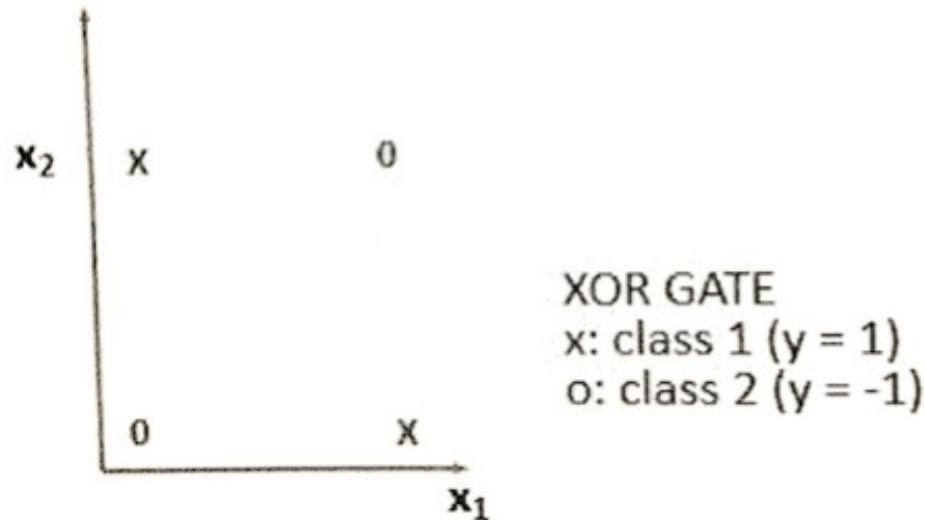
x: class 1 ($y = 1$)

o: class 2 ($y = -1$)

Multi-layer Perceptron

Vimos que el AND y el OR son **linealmente separables** y que el perceptron lo puede reconocer.

Pero también nos dimos cuenta que no todas funciones son separables linealmente, por ejemplo, el XOR:



Lo que podemos hacer, es unir más perceptrones. La estructura resultante puede aprender más funciones con subconjuntos de funciones linealmente separables.

Multi-layer Perceptron

Para hacer lo anterior, debemos introducir transmisiones intermedias entre la capa de entrada y la capa de salida, permitiendo cierta representación interna de la entrada.

Eso se llama MLP.

Multi-layer Perceptron

Para hacer lo anterior, debemos introducir transmisiones intermedias entre la capa de entrada y la capa de salida, permitiendo cierta representación interna de la entrada.

Eso se llama MLP.

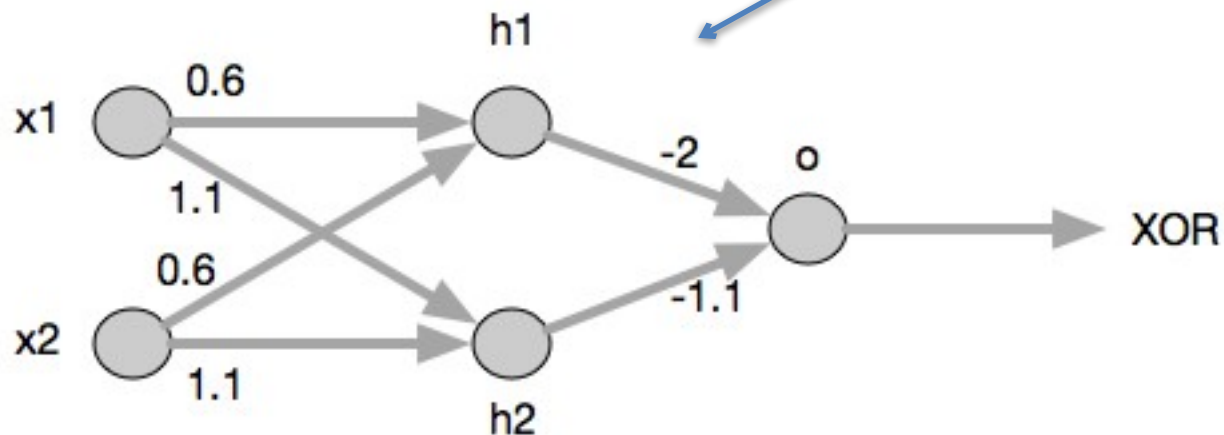
El MLP consiste en al menos 3 capas de nodos: input, hidden y output. Excepto para los nodos de entrada, cada nodo es una NEURONA que usa una FUNCIÓN DE ACTIVACIÓN no lineal.

Así, MLP lo podemos utilizar cuando los datos no son linealmente separables.

Multi-layer Perceptron

Veamos un ejemplo:

x1	x2	y (XOR gate)
1	1	0
1	0	1
0	1	1
0	0	0



Función de activación:

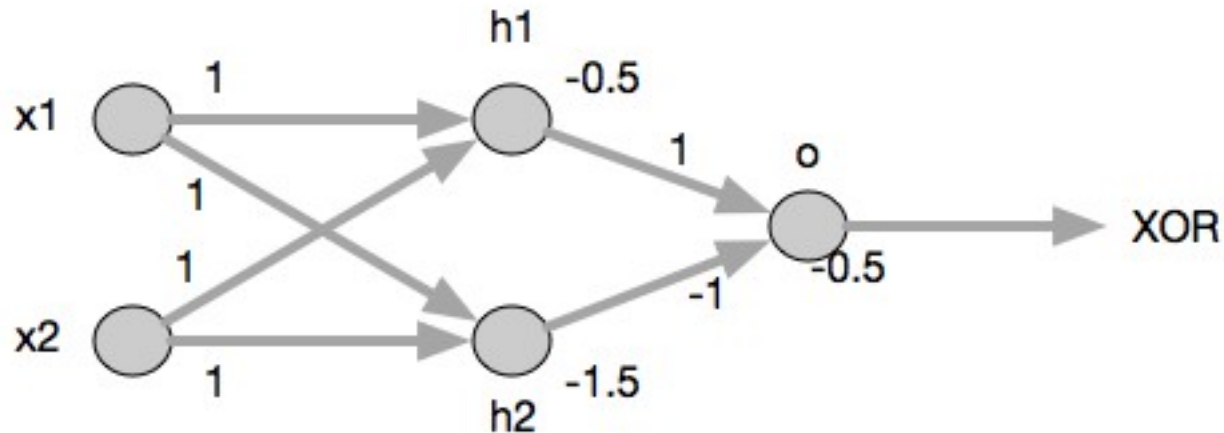
0 si $y < 1$

1 si $y \geq 1$

Multi-layer Perceptron

Veamos un ejemplo:

x1	x2	y (XOR gate)
1	1	0
1	0	1
0	1	1
0	0	0



Actividad en clase:

Lleve a cabo un forward propagation para cada uno de los 4 casos posibles de entrada en x_1 y x_2 y resuelva la salida. Implemente en una planilla excel.